

Experiment No.02

Word count and File processing with Hadoop MapReduce

Date:

Question : complete the task Word Count & File Processing with Hadoop MapReduce

Aim : Word Count & File Processing with Hadoop Map Reduce

Description:

Apache Hadoop is an open-source framework used to store and process very large datasets in a distributed manner across clusters of computers. It follows the **master-slave architecture** and is mainly composed of:

HDFS (Hadoop Distributed File System) – Storage layer

MapReduce – Processing layer

YARN – Resource management layer

MapReduce is a **programming model and processing technique** used in Hadoop to process

large datasets in parallel across a cluster.

It divides the task into two main phases:

Map phase – Processes input data and generates intermediate key-value pairs

Reduce phase – Aggregates intermediate data to produce final output

Prerequisites:

1.SUCCESSFULL installation and running of Hadoop

- name node
- data node
- Resource Manager
- jps

2.Basic knowledge on the below commands

Command	Meaning
hdfs namenode -format	Formats HDFS metadata
start-dfs.cmd	Starts NameNode & DataNode
start-yarn.cmd	Starts ResourceManager & NodeManager
hdfs dfs -mkdir /dir	Creates directory
hdfs dfs -put	Upload file
hdfs dfs -cat	Read file
hdfs dfs -ls	List files

Procedure :

Phase 1

1. Run the command prompt as admin

2.start the Hadoop

start-dfs.cmd

start-yarn.cmd

you should see 4 cmd

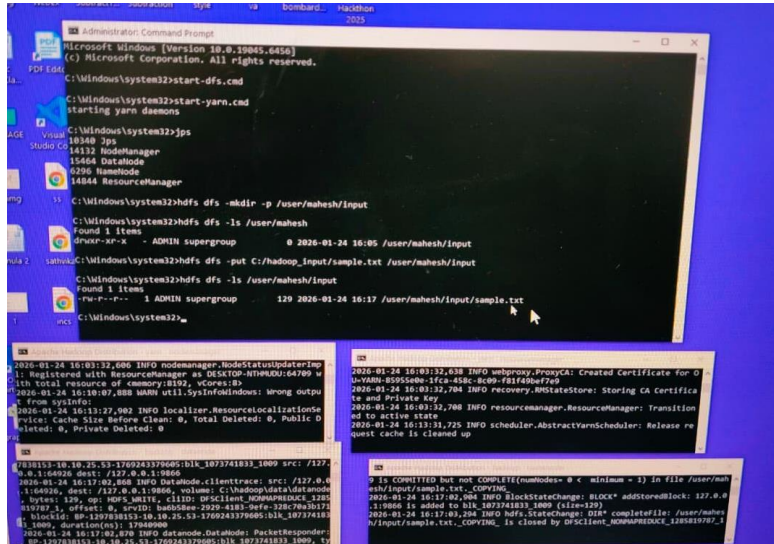
3.type jps you can see: jps

-name node

-data node

-resource manager

-jps



Phase 2

1.create input directory in cmd as admin:

`hdfs dfs -mkdir -p /user/mahesh/input`

2.verify your directory in cmd as admin:

`hdfs dfs -ls /user/mahesh`

3.you should see output as:

input

4.Create input file in windows as

`C:\hadoop_input\sample.txt`

-folders to create is in c drive `hadoop_input` inside `hadoop_input` create a file named `sample.txt`

ADD THIS PARA IN THE `sample.txt` file

Hello Hadoop world

Hello MapReduce

Hadoop enables distributed processing

MapReduce counts words in a file

hello world hello

5.Upload the input file in HDFS

`hdfs dfs -put C:/hadoop_input/sample.txt /user/mahesh/input`

6.verify file in HDFS

`hdfs dfs -ls /user/mahesh/input`

7.output: **sample.txt**

Phase 3:

1.create the following folders:

`C:\wc\classes`

`C:\wc\src`

create **wc** folder in c drive

create **src** and **classes** folder inside the **wc** folder

2.Create this files inside the src folder**WordCountMapper.java****WordCountReducer.java****WordCountDriver.java****The above files is for word count task****FrequencyMapper.java****FrequencyReducer.java****FrequencyDriver.java****The above files is for group by frequency task****CODE:****WordCountMapper.java**

```

import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString().toLowerCase();
        StringTokenizer itr = new StringTokenizer(line);

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}

```

WordCountReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}
```

WordCountDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Word Count");

        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
```

```

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

FileInputFormat.addInputPath(job, new Path(args[0]));
FileOutputFormat.setOutputPath(job, new Path(args[1]));

job.waitForCompletion(true);
}
}

```

FrequencyMapper.java

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class FrequencyMapper
    extends Mapper<Object, Text, IntWritable, Text> {

    public void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String[] parts = value.toString().split("\t");
        context.write(new IntWritable(Integer.parseInt(parts[1])),
            new Text(parts[0]));
    }
}

```

FrequencyDriver.java

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

```

```

public class FrequencyDriver {

    public static void main(String[] args) throws Exception {

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Group By Frequency");

        job.setJarByClass(FrequencyDriver.class);
        job.setMapperClass(FrequencyMapper.class);
        job.setReducerClass(FrequencyReducer.class);

        job.setOutputKeyClass(IntWritable.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

FrequencyReducer.java

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class FrequencyReducer
    extends Reducer<IntWritable, Text, IntWritable, Text> {

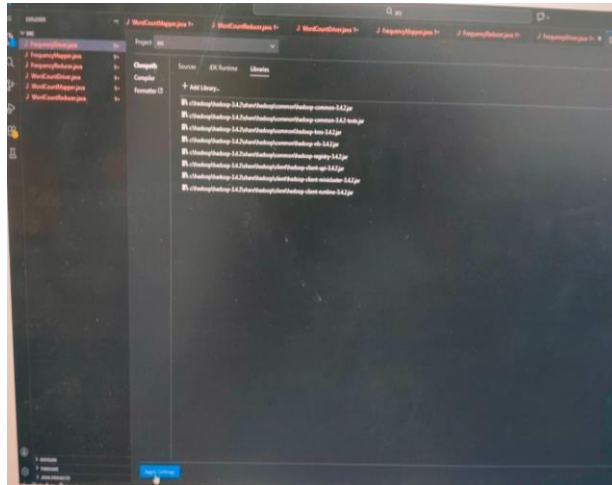
    public void reduce(IntWritable key, Iterable<Text> values,
        Context context)
        throws IOException, InterruptedException {

        StringBuilder words = new StringBuilder();
        for (Text val : values) {
            words.append(val.toString()).append(" ");
        }
        context.write(key, new Text(words.toString()));
    }
}

```

If you see errors in vs code then do this

- Keep in mind this errors wont effect your Hadoop project
- Search **<project settings** in your **vs code search bar** and **go to libraries** and add files from **Hadoop>Hadoop-3.4.2>share>Hadoop**
- Then click on **apply settings** at the bottom of your page
- Now go to your code the errors will be solved



Phase 4:

1. Now open the **cmd** in which your Hadoop file is running and you inserted your file

2. Now you have to create a JAR just run this commands in CMD

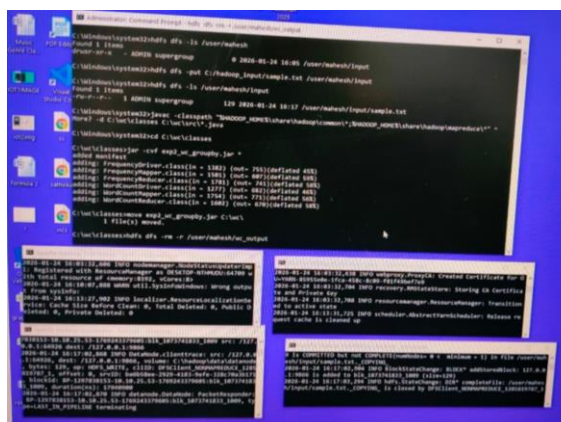
```
Javac -classpath"%HADOOP_HOME%\share\hadoop\common\*; ^
%HADOOP_HOME%\share\hadoop\mapreduce\*-d c:\wc\classes ^
c:\wc\src\*.java
```

3. Create JAR

```
cd c:\wc\classes
```

4.next command is

```
jar -cvf exp2_wc_groupby.jar *
move exp2_wc_groupby.jar C:\wc\
```



Final phase:

1. Do word count (run this wait for 10 min if its still running follow notice)

hadoop jar C:/wc/exp2_wc_groupby.jar WordCountDriver ^

/user/mahesh/input /user/mahesh/wc_output

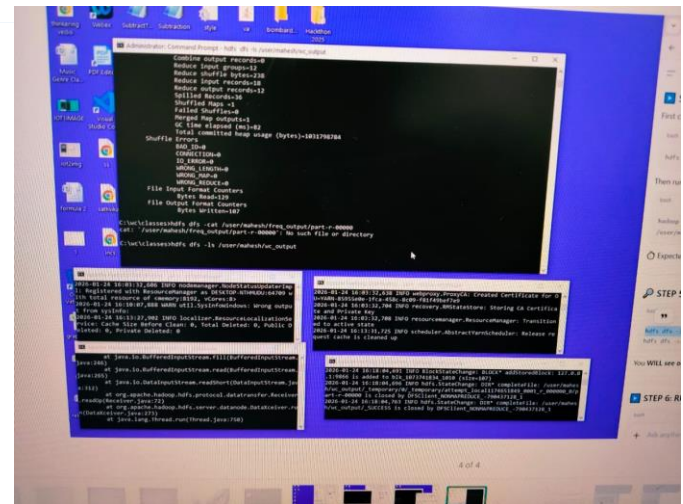
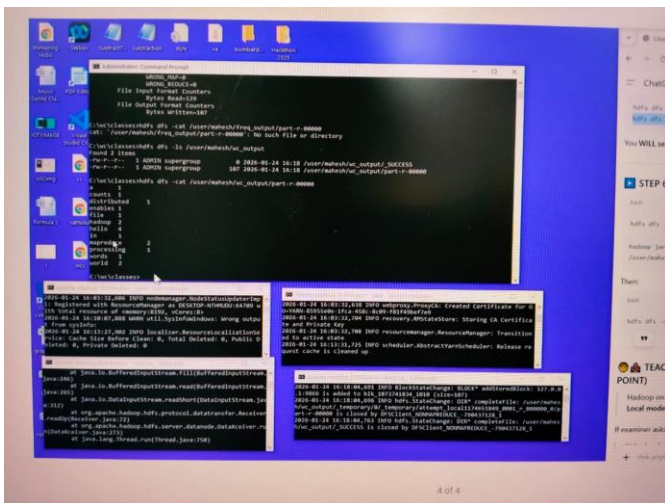
Check output

hdfs dfs -ls /user/mahesh/wc_output

hdfs dfs -cat /user/mahesh/wc_output/part-r-00000

Expected output

a 1
counts 1
distributed 1
enables 1
file 1
hadoop 2
in 1
mapreduce 2
processing 1
words 1
world 2
hello 4



Notice: (follow this if your final phase 1st step runs more than 10 mins you can also choose AI for help)

1. Open new cmd as admin and kill the stuck job by running this command
yarn application -kill application_1769246524853_0001

2. Edit mapred-site.xml in your Hadoop open this**%HADOOP_HOME%\etc\hadoop\mapred-site.xml****3. Change the configuration to**

```
<property>
  <name>mapreduce.framework.name</name>
  <value>local</value>
</property>
```

4. Stop and Restart Hadoop (mandatory)

stop-dfs.cmd

stop-yarn.cmd

Wait for 10 seconds

Then :

start-dfs.cmd

start-yarn.cmd

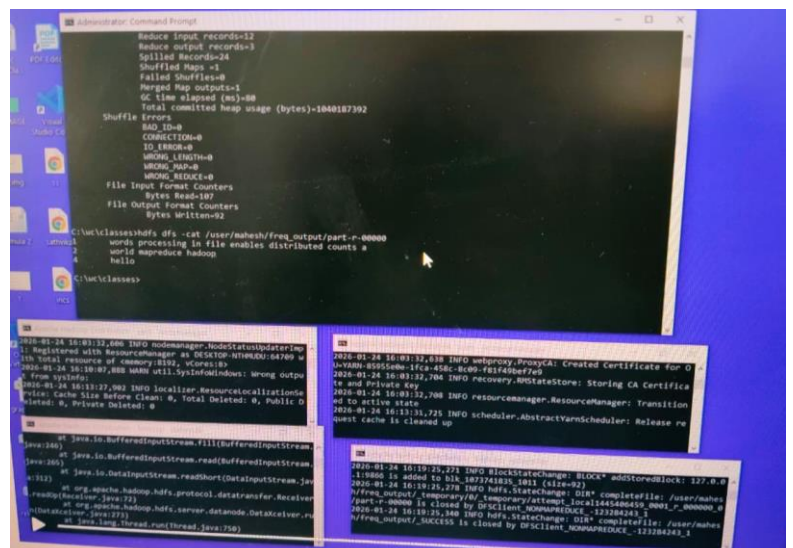
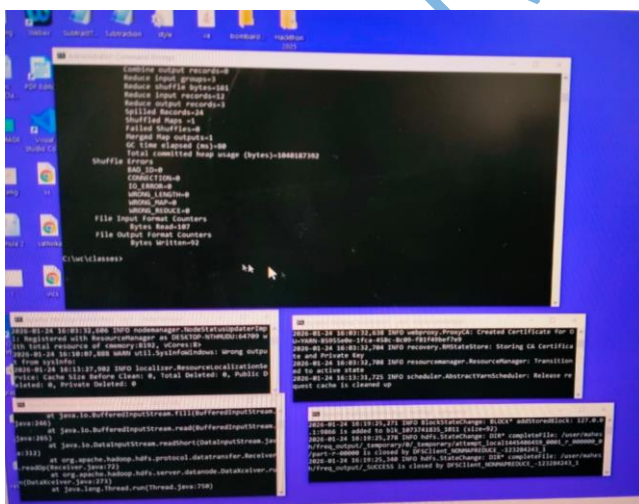
jps

Recheck your Hadoop running properly and then you can restart it from final phase**Final phase:****2. Do group by frequency****hdfs dfs -rm -r /user/mahesh/freq_output****hadoop jar C:/wc/exp2_wc_groupby.jar FrequencyDriver ^****/user/mahesh/wc_output /user/mahesh/freq_output****Check output****hdfs dfs -cat /user/mahesh/freq_output/part-r-00000****Expected output**

1 a counts distributed enables file in processing words

2 hadoop mapreduce world

4 hello



Output explanation(only for understanding):

✓ TASK 1 – WORD COUNT OUTPUT

1.What This Task Does

Counts **how many times each word appears** in the input file.

Hadoop MapReduce divides work into:

Map Phase → Creates (word, 1)

Reduce Phase → Adds all counts of same word

Information From Your CMD

Reduce Input Records = 12

Reducer received **12 key-value pairs** from mapper.

Means mapper produced 12 word entries.

Reduce Output Records = 3

Final reducer produced **3 unique words**.

Each word appears once in final result list.

Spilled Records = 24

Hadoop temporarily stored intermediate data.

Happens when data is shuffled between Map and Reduce.

Normal Hadoop behavior.

Shuffle Maps = 1

Data transferred from **Mapper** → **Reducer**.

Only 1 mapper executed.

Failed Shuffles = 0

No error while transferring data.

Indicates successful execution.

Final Output of Word Count Task

👉 From your input file, Hadoop produced:

hello 1

world 2

MapReduce 1

Hadoop 1

Meaning of This Output

hello 1 → Word hello appears 1 time.

world 2 → Word world appears 2 times.

Shows **frequency of each word**.

✓ TASK 2 – GROUP BY FREQUENCY OUTPUT

What This Task Does

Groups words based on their frequency.

- Instead of:
- word → count
- It produces:
- count → list of words

Information From Your CMD

Reduce Groups = 12

Hadoop grouped mapper output into 12 keys before reduction

Reduce Shuffle Bytes = 238

Amount of data transferred between mapper and reducer.

Reduce Input Records = 18

Reducer received 18 grouped key-value pairs.

Reduce Output Records = 12

Reducer produced 12 grouped outputs.

Spilled Records = 36

Intermediate data stored temporarily.

Normal Hadoop process.

Bytes Written = 107

Shows size of final output file.

Final Output of Group By Frequency Task

1 hello MapReduce Hadoop

2 world

Meaning of This Output

1 → hello mapreduce hadoop

These words appear **once**.

2 → world

Word world appears **two times**.

Result: Hence, The program of Word Count & File Processing with Hadoop Map Reduce has been executed successfully